

Sistema de pontuação FotoMac

Como comparar modelos de reconhecimento facial com uma nota só

Thiago José Nunes

23 de setembro de 2026

1. Para que serve

A bancada existe para escolher **um** modelo de reconhecimento facial para o FOTOMAC. Hoje o candidato é YuNet + SFace, rodando local e de graça. Amanhã entram outros — InsightFace, DeepFace, ou uma API paga da AWS, Azure ou Google.

Para escolher entre eles é preciso uma **nota**, calculada do mesmo jeito para todos. Este documento define essa nota.

A regra que torna tudo isso possível: **sem gabarito não existe acerto**. Gabarito é alguém dizendo, antes do teste, em quais fotos cada pessoa de fato aparece. Sem isso o número "achou 12 fotos" não diz se as 12 estão certas.

2. O gabarito

Arquivo gabarito.json, na raiz do projeto. É texto, editável à mão:

```
{
  "acervo": "fotos",
  "acervo_completo": true,
  "pessoas": {
    "thiago": {
      "consultas": ["IMG_1128.JPG", "IMG_1129.JPG", "IMG_1130.JPG"],
      "aparece_em": ["IMG_1128.JPG", "IMG_1129.JPG", "IMG_1130.JPG"]
    }
  }
}
```

- **consultas** — as fotos que podem ser usadas como autorretrato dessa pessoa.
- **aparece_em** — todas as fotos do acervo em que ela realmente aparece.
- **acervo_completo: true** — declara que toda foto **fora** dessa lista foi conferida e a pessoa não está nela. É essa declaração que permite contar erro. Sem ela, uma foto não listada poderia ser um acerto que ninguém marcou.

Para acrescentar uma pessoa, basta mais uma entrada em pessoas. Nada no código muda.

3. As quatro medidas

3.1 Os quatro resultados possíveis

Para cada foto do acervo, o modelo diz "é ela" ou "não é". O gabarito diz a verdade. Dá quatro combinações:

	Gabarito diz é ela	Gabarito diz não é
Modelo marcou	Acerto (TP)	Falso positivo (FP)
Modelo não marcou	Perdida (FN)	Rejeição correta (TN)

Os dois erros doem de jeitos diferentes, e é importante entender a diferença:

- **Falso positivo** — o visitante recebe a foto de um estranho. É o erro grave: mostra a foto de uma pessoa para outra. Num sistema com fotos de formatura, é problema de privacidade, não só de qualidade.
- **Perdida** — o visitante não recebe uma foto que era dele. Chateia, mas não expõe ninguém.

3.2 Precisão, recall e F1

- **Precisão** = dos que o modelo marcou, quantos estavam certos. Precisão baixa = entrega foto de estranho.
- **Recall** = das que existiam, quantas ele achou. Recall baixo = deixa foto para trás.
- **F1** = a média harmônica das duas. Vale 1,0 só quando as duas estão altas.

Usa-se F1, e não "taxa de acerto", porque acerto simples engana: um modelo que nunca marca nada acerta 201 das 204 fotos — e é inútil.

3.3 Separação (σ) — a medida que quase ninguém usa e que decide tudo

F1 igual a 1,0 pode ser sorte. Pode ser que o pior acerto tenha dado 0,52 e o melhor erro 0,51: o modelo acertou tudo hoje, e a primeira foto nova quebra.

A **separação** mede a folga. Ela responde: *quantos desvios-padrão o pior acerto está acima da média dos erros?*

$$\sigma = \frac{\text{pior acerto} - \text{média dos erros}}{\text{desvio-padrão dos erros}}$$

Por que dividir pelo desvio-padrão: assim o número fica **adimensional**. O SFace devolve cosseno de -1 a 1 ; uma API paga pode devolver "confiança" de 0 a 100 . Uma diferença de $0,15$ num e de 15 no outro podem significar a mesma coisa. Dividindo pela dispersão dos próprios erros, os dois viram comparáveis.

3σ já é folga confortável — é a nota cheia.

3.4 Tempo

Duas medidas separadas, porque acontecem em momentos diferentes:

- **Busca** — o visitante está parado na frente do totem. Orçamento: 100 ms é nota cheia, 3000 ms é zero.
- **Indexação por foto** — roda de madrugada, sem ninguém esperando. Orçamento: 0,10 s por foto é nota cheia, 2,00 s é zero.

Entre o bom e o inaceitável a nota cai em escala **logarítmica**, não linear: para uma pessoa, 50 ms e 100 ms são indistinguíveis, mas 1 s e 2 s não são.

4. A nota: 100 pontos

Componente	Pontos	O que mede
Acerto	60	F1 médio das consultas
Separação	20	folga em σ , saturando em 3σ
Busca	10	milissegundos por consulta
Indexação	10	segundos por foto

Por que esses pesos

Acerto vale 60 porque é a razão de existir do sistema. Um modelo rápido que erra não serve para nada; um modelo lento que acerta ainda dá para usar.

Separação vale 20 porque F1 sozinho é frágil. Dois modelos com $F1 = 1,0$ não são equivalentes: o que tem 5σ de folga vai continuar funcionando com fotos novas; o que tem $0,3\sigma$ vai quebrar. Esses 20 pontos são o que separa "funcionou no teste" de "vai funcionar em produção".

Tempo vale 20 no total porque nesta escala (centenas a milhares de fotos) quase qualquer modelo moderno passa. Peso maior faria a nota premiar velocidade que não é gargalo.

Os pesos ficam no topo de `scoring.py`. Mudar é legítimo — desde que não se mude **no meio de uma comparação**, porque aí os modelos deixam de ser comparáveis.

Cada modelo é avaliado no SEU melhor limiar

Isto é essencial para comparar coisas diferentes. O 0,363 do SFace não quer dizer nada para uma API que devolve 0 a 100. Cada modelo é medido no limiar que maximiza o F1 **dele**; a nota compara a qualidade da separação, não a escala dos números.

O que ficou de fora da nota, de propósito

Custo. Um modelo pago pode ser melhor e ainda assim não caber no orçamento. Isso é decisão de negócio, não medida de qualidade — misturar as duas coisas esconde a escolha em vez de explicá-la. O custo entra na tabela de comparação, em R\$ por mil fotos, **ao lado** da nota, nunca dentro dela.

5. Resultado do primeiro candidato

YuNet + SFace (OpenCV), local, gratuito. Nota 97,9 / 100.

Componente	Nota	Medida
Acerto	60,0 / 60	F1 = 1,000, precisão 1,000, recall 1,000
Separação	20,0 / 20	5,07 σ
Busca	10,0 / 10	0,41 ms
Indexação	7,9 / 10	0,19 s por foto

Cobertura do acervo: **3583 rostos em 204 de 204 fotos** — nenhuma foto ficou de fora.

Por consulta, no limiar 0,45 — 204 fotos, 3583 rostos:

Consulta	Acertos	Falsos positivos	Perdidas
IMG_1128.JPG	2	0	0
IMG_1129.JPG	2	0	0
IMG_1130.JPG	2	0	0

O ajuste que foi feito — e por quê

O limiar padrão do SFace, **0,363**, **erra neste acervo**. Com IMG_1129 como consulta, a foto IMG_3011 dá 0,3728 e passa: é um rapaz diferente, de cabelo e bigode parecidos. Falso positivo.

Medindo o conjunto todo:

- **pior acerto** (a foto correta de menor similaridade): **0,5202**
- **melhor erro** (a foto errada de maior similaridade): **0,3728**

Como o pior acerto está **acima** do melhor erro, existe uma faixa inteira de limiares que acerta tudo e não erra nada. **0,45** é o ponto médio: o valor mais distante das duas bordas, com cerca de 0,07 de folga de cada lado. É o novo COSINE_THRESHOLD.

O segundo ajuste: levar TODOS os rostos à comparação

Na primeira rodada, os filtros de qualidade (MIN_DET_SCORE=0,60, MIN_FACE_SIDE=40) descartavam rosto pequeno ou pouco nítido antes de gerar vetor. Resultado: 1004 rostos, e **7 fotos ficavam sem rosto nenhum**.

Isso é um problema real para o produto: numa foto de formatura o visitante aparece ao fundo, pequeno. Rosto descartado é foto perdida.

Medido, com o gabarito, o efeito de abrir os filtros:

MIN_DET_SCORE	MIN_FACE_SIDE	Rostos	Fotos cobertas	Pior acerto	Melhor erro
0,60	40	1004	197	0,5202	0,3728
0,50	10	3583	204	0,5202	0,3809
0,40	10	3999	204	0,5202	0,3809
0,30	10	4607	204	0,0546	0,5122

Adotado: **0,50 e 10**. Triplica os rostos, cobre todas as fotos, e o melhor erro sobe só de 0,3728 para 0,3809 — continua bem longe do limiar 0,45.

Antes de baixar, conferi se rosto minúsculo não vira vetor sem sentido: a similaridade média entre rostos de 10-19 px é +0,16, contra +0,12 entre rostos grandes. Não é degenerado.

MIN_DET_SCORE=0,30 quebra tudo e vale entender por quê, porque é um bug de projeto e não de parâmetro: nesse nível o detector inventa uma caixa espúria e **grande** no autorretrato — em IMG_1128, área 356206 contra 319414 do rosto real. Como a regra de escolha do rosto da consulta é "maior área", a caixa falsa vence e o vetor da busca vira lixo. O problema está no lado da **consulta**, não do acervo.

O ajuste que NÃO foi feito — e por quê

Varri 60 combinações de MAX_SIDE, MIN_DET_SCORE e MIN_FACE_SIDE usando o gabarito como alvo. A combinação com maior margem foi MAX_SIDE=1280, MIN_FACE_SIDE=150, com margem 0,29 contra 0,15 da atual.

Rejeitada. Essa configuração guarda **9 rostos no acervo inteiro de 204 fotos**. Ela ganha margem porque joga fora quase tudo — inclusive os rostos que o sistema real precisaria encontrar. Num acervo de formatura, o visitante aparece ao fundo, pequeno. Otimizar até o gabarito ficar perfeito, à custa do problema real, é **ajustar ao gabarito**, não melhorar o modelo. Os parâmetros de detecção ficam como estão.

Vale registrar o contraexemplo: MAX_SIDE=2560 **piora muito** (margem -0,26). Resolução maior não é melhor aqui.

6. A ressalva honesta

O gabarito tem **uma pessoa e três fotos positivas**. Isso é suficiente para afirmar:

- que 0,363 **erra** neste acervo (basta um contraexemplo para derrubar um valor);
- que existe folga entre acertos e erros neste conjunto;
- que 0,45 é **melhor** que 0,363 aqui.

Não é suficiente para afirmar que 0,45 é o valor certo em geral. Recall medido em duas fotos por consulta tem pouca força estatística.

O próximo passo que mais aumenta a confiança do TCC: pedir a mais duas ou três pessoas que apareçam nas fotos do evento um autorretrato, marcar em quais fotos cada uma aparece, e

recalibrar. Com 4 pessoas e algumas dezenas de fotos positivas, os números passam a sustentar uma afirmação geral.

7. Como rodar

```
# pontua o modelo atual contra o gabarito
.venv/bin/python benchmark.py \
  --consulta fotos/IMG_1128.JPG \
  --consulta fotos/IMG_1129.JPG \
  --consulta fotos/IMG_1130.JPG
```

A nota sai no terminal, no resultados/<data>_resumo.json e no topo da página visual.

Para testar outro modelo, nomeie-o para a tabela de comparação:

```
.venv/bin/python benchmark.py --modelo "InsightFace buffalo_l" --consulta ...
```

8. Como plugar um modelo novo

O sistema de pontuação não sabe nada sobre YuNet ou SFace. Ele só precisa de uma lista (nome_do_arquivo, similaridade) por consulta. Qualquer modelo que produza isso é pontuável.

Para encaixar um candidato novo, escreva um substituto de `fotomac_face.FaceEngine` com dois métodos:

Método	Recebe	Devolve
<code>index_image(caminho)</code>	uma foto do acervo	lista de rostos, cada um com embedding e bbox
<code>query_face(caminho)</code>	o autorretrato	um rosto, escolhido por regra determinística

O resto — varredura, cronometragem, ranking, gabarito, nota, página visual — continua igual. É por isso que `benchmark.py` é uma camada **por cima** do motor, e não dentro dele.

Para uma API paga que não devolve vetor, e sim um "grau de confiança" entre duas fotos, troque a etapa de comparação por chamadas à API e alimente o mesmo ranking. A nota continua válida: ela é calculada sobre o ranking, não sobre a escala.

9. Captura pela webcam

`capture.py` simula o totem: a pessoa olha para a câmera e o sistema só deixa tirar quando o rosto está bom.

```
.venv/bin/python capture.py          # janela ao vivo; ESPAÇO tira, ESC
cancela
.venv/bin/python capture.py --sem-janela # conta 3,2,1 e escolhe o melhor
quadro
.venv/bin/python capture.py --buscar    # tira e já roda a busca no acervo
```

O portão de qualidade recusa, com a instrução em vez do erro técnico:

Situação	Mensagem
Sem rosto	"Nenhum rosto: chegue mais perto e olhe para a camera"
Confiança abaixo de 0,50	"Rosto pouco nitido: procure mais luz"
Rosto menor que 80 px	"Rosto pequeno: chegue mais perto"
Mais de um rosto	"Mais de um rosto no quadro: fique sozinho"

O limite de 80 px aqui é maior que o `MIN_FACE_SIDE=10` do acervo, de propósito: nas fotos do evento aceitamos o rosto que houver, porque a foto é o que é; na frente da câmera a pessoa pode simplesmente chegar mais perto.

No macOS a câmera exige permissão em **Ajustes do Sistema > Privacidade e Segurança > Câmera**, para o aplicativo de onde o comando roda, e é preciso fechar e reabrir esse aplicativo depois de liberar.

10. Tabela de comparação

Preencher conforme os candidatos forem testados. Esta é a tabela que vai para o TCC.

Modelo	Nota	F1	σ	Busca	Indexação	Custo / 1000 fotos
YuNet + SFace (OpenCV)	97,9	1,000	5,07	0,41 ms	0,19 s	R\$ 0 (local)
<i>(próximo candidato)</i>						